

Projets Tutorés

Master 1 – Cybersécurité

Année universitaire 2025–2026

Enseignant : Dr. Tegawendé F. Bissyandé

Université Joseph Ki-Zerbo / CyLab

Document décrivant quatre projets tutorés proposés aux étudiants de Master 1.
Deux projets portent sur l'analyse de la surface d'attaque réseau du Burkina Faso
à l'aide de la suite ZMap. Deux projets portent sur la cybersécurité logicielle.

Table des matières

1	Présentation générale	2
2	Projet 1 – Cartographie de l’espace IP burkinabè et évaluation de l’exposition des services réseau	3
2.1	Fiche synthétique	3
2.2	Contexte et motivation	3
2.3	Objectifs du projet	3
2.4	Méthodologie	3
2.5	Livrables attendus	4
2.6	Considérations éthiques et légales	4
3	Projet 2 – Audit de l’écosystème HTTPS/TLS des serveurs web burkinabè	5
3.1	Fiche synthétique	5
3.2	Contexte et motivation	5
3.3	Objectifs du projet	5
3.4	Méthodologie	5
3.5	Livrables attendus	6
3.6	Considérations éthiques et légales	6
4	Projet 3 – Détection automatisée de vulnérabilités dans des projets open source par analyse statique	7
4.1	Fiche synthétique	7
4.2	Contexte et motivation	7
4.3	Objectifs du projet	7
4.4	Méthodologie	8
4.5	Livrables attendus	8
5	Projet 4 – Audit de la chaîne d’approvisionnement logicielle : analyse des dépendances et détection de composants vulnérables	9
5.1	Fiche synthétique	9
5.2	Contexte et motivation	9
5.3	Objectifs du projet	9
5.4	Méthodologie	9
5.5	Livrables attendus	10
5.6	Compétences développées	10
6	Récapitulatif et modalités pratiques	11
6.1	Modalités d’évaluation	11
6.2	Ressources et références	11

Présentation générale

Ce document décrit quatre projets tutorés destinés aux étudiants de première année de Master. Chaque projet est conçu pour être réalisé par un groupe de 3 à 5 étudiants sur une durée d'un mois (environ 2 à 3 semaines de travail effectif). Les projets sont organisés en deux axes thématiques complémentaires.

Axe 1 – Analyse de la surface d'attaque réseau (Projets 1 et 2). Ces projets s'appuient sur la suite d'outils ZMap développée par l'Université du Michigan. Ils visent à produire une cartographie empirique de l'espace Internet burkinabè et à évaluer sa posture de sécurité. Les résultats pourront être transmis aux autorités compétentes (ANSSI-BF, CIRT.BF, ARCEP) sous forme de rapport technique.

Axe 2 – Cybersécurité logicielle (Projets 3 et 4). Ces projets portent sur l'analyse de la sécurité du code source et des dépendances logicielles. Ils mobilisent des techniques d'analyse statique, de détection de vulnérabilités et d'audit de la chaîne d'approvisionnement logicielle (*software supply chain security*). Ces compétences sont aujourd'hui centrales dans l'industrie du développement logiciel sécurisé.

TABLE 1 – Vue d'ensemble des quatre projets tutorés.

#	Axe	Intitulé	Outils principaux
1	Réseau	Cartographie de l'espace IP burkinabè	ZMap, LZR, ZAnnotate
2	Réseau	Audit HTTPS/TLS des serveurs web .bf	ZMap, ZGrab2, ZLint
3	Logiciel	Détection de vulnérabilités dans des projets open source	CodeQL, Semgrep
4	Logiciel	Audit de la chaîne d'approvisionnement logicielle	OSV, Syft, Gype

Projet 1 – Cartographie de l'espace IP burkinabè et évaluation de l'exposition des services réseau

Fiche synthétique

Intitulé : Cartographie de l'espace IP burkinabè et évaluation de l'exposition des services réseau

Mots-clés : ZMap, LZR, ZAnnotate, scan réseau, surface d'attaque, Burkina Faso

Nombre d'étudiants : 3 à 5

Durée : 1 mois (2–3 semaines)

Prérequis : Réseaux TCP/IP, systèmes Linux, notions de sécurité réseau

Contexte et motivation

Le Burkina Faso dispose actuellement de 3 centres de données et de 2 points d'échange Internet (IXP), avec plusieurs fournisseurs d'accès (Orange Burkina Faso, ONATEL, etc.) possédant des préfixes ASN enregistrés auprès de l'AFRINIC. Malgré cette infrastructure croissante, il n'existe aucune étude publique systématique de la surface d'attaque réellement exposée sur l'espace IP national. Des services potentiellement dangereux (Telnet, RDP, VNC sans authentification) peuvent être accessibles depuis Internet sans que les opérateurs en soient conscients.

Ce projet vise à produire la première cartographie de référence de l'espace IP burkinabè, en identifiant les hôtes accessibles et les services exposés. Les résultats constitueront une base de données exploitable par les institutions nationales de cybersécurité.

Objectifs du projet

- O1.** Énumérer l'ensemble des préfixes IP alloués au Burkina Faso à partir des données WHOIS de l'AFRINIC.
- O2.** Réaliser un scan réseau à l'aide de ZMap sur les ports les plus courants (22, 23, 25, 80, 443, 3306, 3389, 5900, 8080, etc.).
- O3.** Utiliser LZR pour identifier les services répondant sur des ports non standard (LZR identifie 99 % des services inattendus en cinq poignées de main).
- O4.** Enrichir les résultats avec des métadonnées de routage et de géolocalisation via ZAnnotate.
- O5.** Classifier les services découverts par catégorie (serveurs web, panneaux d'administration, services legacy, bases de données exposées).
- O6.** Produire un rapport technique avec recommandations à destination de l'ANSSI-BF.

Méthodologie

Le projet se décompose en quatre phases principales :

- Phase 1. Collecte des préfixes IP** (jours 1–2). Extraction des blocs IP alloués au Burkina Faso via les bases WHOIS de l'AFRINIC et les données RIR. Validation croisée avec les annonces BGP visibles dans les *route collectors* publics (RIPE RIS, RouteViews).
- Phase 2. Scan réseau et identification des services** (jours 3–6). Déploiement de ZMap pour scanner les ports cibles. Configuration de LZR pour effectuer l'identification applicative (*application-layer fingerprinting*) sur les hôtes répondants. Documentation rigoureuse de la méthodologie de scan (débit, durée, paramètres).

Phase 3. Enrichissement et analyse (jours 7–10). Utilisation de ZAnnotate pour ajouter les informations ASN, opérateur et géolocalisation. Analyse statistique des résultats : distribution des services par port, par opérateur, par type. Identification des expositions critiques.

Phase 4. Rédaction et restitution (jours 11–14). Rédaction du rapport technique. Élaboration de visualisations (cartes, diagrammes). Préparation de la soutenance.

Livrables attendus

- Base de données structurée des hôtes et services découverts (format CSV/JSON).
- Rapport technique (20–30 pages) incluant méthodologie, résultats, analyse et recommandations.
- Jeu de scripts reproductibles (pipeline de scan et d’analyse).
- Présentation de soutenance.

Considérations éthiques et légales

Le scan réseau doit être mené dans un cadre éthique strict. Les étudiants devront :

- Obtenir les autorisations nécessaires avant tout scan actif.
- Limiter le débit de scan pour ne pas perturber les services en production.
- Ne pas tenter d’exploiter les vulnérabilités découvertes.
- Anonymiser les données dans le rapport public (pas d’adresses IP en clair).
- Prévoir un processus de divulgation responsable auprès des opérateurs concernés.

Projet 2 – Audit de l'écosystème HTTPS/TLS des serveurs web burkinabè

Fiche synthétique

Intitulé : Audit de l'écosystème HTTPS/TLS des serveurs web burkinabè

Mots-clés : ZMap, ZGrab2, ZLint, TLS, HTTPS, certificats X.509, Burkina Faso

Nombre d'étudiants : 3 à 5

Durée : 1 mois (2–3 semaines)

Prérequis : Réseaux, protocoles de sécurité (TLS/SSL), cryptographie de base

Contexte et motivation

Avec la multiplication des services en ligne au Burkina Faso (banques mobiles, portails e-gouvernement, plateformes universitaires), la sécurité des communications HTTPS est devenue un enjeu critique. Or, la majorité des déploiements TLS dans le monde souffrent de configurations inadéquates : versions obsolètes du protocole (TLS 1.0/1.1), suites cryptographiques faibles (RC4, NULL, EXPORT), certificats expirés ou auto-signés, absence de *forward secrecy*. Ce problème est très peu étudié en Afrique de l'Ouest.

Ce projet propose d'évaluer quantitativement la posture de sécurité TLS de l'ensemble des serveurs web accessibles dans l'espace IP burkinabè, en utilisant la chaîne d'outils ZMap, ZGrab2 et ZLint.

Objectifs du projet

- O1.** Identifier tous les hôtes répondant sur le port TCP/443 dans l'espace IP du Burkina Faso via ZMap.
- O2.** Effectuer des *handshakes* TLS complets avec ZGrab2 pour collecter : versions de protocole supportées, suites cryptographiques proposées, chaînes de certificats, en-têtes HSTS.
- O3.** Auditer la conformité des certificats X.509 aux standards CA/Browser Forum avec ZLint.
- O4.** Classifier les résultats : utilisation de TLS 1.0/1.1 (obsolète), suites faibles, certificats auto-signés ou expirés, absence de *forward secrecy*, présence/absence de HSTS.
- O5.** Comparer les résultats aux benchmarks globaux issus de Censys ou d'études pays publiées.
- O6.** Produire des recommandations pour l'ARCEP et l'ANSSI-BF.

Méthodologie

- Phase 1. Découverte des serveurs HTTPS** (jours 1–3). Scan ZMap sur TCP/443 de l'espace IP burkinabè. Construction de la liste des hôtes HTTPS actifs.
- Phase 2. Collecte des données TLS** (jours 4–7). Déploiement de ZGrab2 pour effectuer les *handshakes* TLS. Collecte systématique des paramètres cryptographiques et des certificats. Passage des certificats dans ZLint pour l'audit de conformité.
- Phase 3. Analyse et classification** (jours 8–11). Catégorisation des serveurs par niveau de sécurité TLS. Analyse sectorielle (gouvernement, banque, santé, éducation). Comparaison avec les données globales.
- Phase 4. Rapport et recommandations** (jours 12–14). Rédaction du rapport avec matrices de risques. Élaboration de recommandations concrètes (standards TLS minimaux, politique de certificats). Soutenance.

Livrables attendus

- Base de données des configurations TLS collectées (format JSON structuré).
- Rapport d’audit (20–30 pages) avec analyse sectorielle et comparaison internationale.
- Tableau de bord de synthèse (statistiques clés, visualisations).
- Scripts et pipeline d’audit reproductibles.
- Note de recommandations à destination des régulateurs.

Considérations éthiques et légales

Les mêmes précautions que le Projet 1 s’appliquent. En particulier :

- Les *handshakes* TLS effectués par ZGrab2 sont passifs (pas d’exploitation).
- Les certificats collectés sont des données publiques par conception.
- Les résultats identifiant des configurations dangereuses doivent faire l’objet d’une communication responsable aux opérateurs concernés.

Projet 3 – Détection automatisée de vulnérabilités dans des projets open source par analyse statique

Fiche synthétique

Intitulé : Détection automatisée de vulnérabilités dans des projets open source par analyse statique

Mots-clés : analyse statique, CodeQL, Semgrep, CWE, vulnérabilités logicielles, open source

Nombre d'étudiants : 3 à 5

Durée : 1 mois (2–3 semaines)

Prérequis : Programmation (Python, Java ou C/C++), notions de génie logiciel, bases de sécurité applicative

Contexte et motivation

Les vulnérabilités logicielles constituent l'un des vecteurs d'attaque les plus exploités en cybersécurité. Les bases de données publiques telles que la *National Vulnerability Database* (NVD) et le système *Common Weakness Enumeration* (CWE) répertorient des dizaines de milliers de failles découvertes chaque année dans des logiciels largement déployés. L'analyse statique du code source — c'est-à-dire l'examen automatisé du code sans l'exécuter — est une technique fondamentale pour détecter ces vulnérabilités en amont, avant la mise en production.

Des outils modernes comme CodeQL (développé par GitHub/Microsoft) et Semgrep (développé par Semgrep Inc.) permettent d'écrire des requêtes personnalisées pour rechercher des patterns de code vulnérables. CodeQL modélise le code sous forme de base de données relationnelle interrogeable, tandis que Semgrep utilise des patterns syntaxiques proches du code source. Ces outils sont gratuits pour les projets open source et largement adoptés dans l'industrie.

Ce projet propose aux étudiants d'appliquer ces outils à un corpus de projets open source populaires afin d'évaluer leur capacité de détection, de comparer leurs performances, et potentiellement de découvrir des vulnérabilités non encore répertoriées.

Objectifs du projet

- O1.** Constituer un corpus de 5 à 10 projets open source populaires (par exemple : applications web en Python/Django, bibliothèques Java, outils système en C) présentant un historique de CVE documenté.
- O2.** Déployer CodeQL et Semgrep sur ce corpus et exécuter les jeux de règles standards (OWASP Top 10, CWE Top 25).
- O3.** Analyser les résultats : taux de vrais positifs, faux positifs, types de vulnérabilités détectées (injection SQL, XSS, *buffer overflow*, désérialisation non sécurisée, etc.).
- O4.** Comparer les deux outils en termes de couverture, précision, temps d'exécution et facilité d'utilisation.
- O5.** Écrire au moins 3 règles personnalisées (queries CodeQL ou règles Semgrep) ciblant des patterns spécifiques non couverts par les règles par défaut.
- O6.** Documenter toute vulnérabilité potentiellement nouvelle découverte et préparer un rapport de divulgation responsable si applicable.

Méthodologie

- Phase 1. Étude bibliographique et sélection du corpus** (jours 1–3). Revue des travaux sur l’analyse statique de sécurité. Sélection des projets cibles sur GitHub en fonction de critères objectifs (popularité, langage, historique CVE). Installation et prise en main de CodeQL et Semgrep.
- Phase 2. Exécution des analyses** (jours 4–7). Construction des bases de données CodeQL pour chaque projet. Exécution des suites de requêtes de sécurité. Exécution parallèle de Semgrep avec les *rulesets* communautaires. Collecte systématique des résultats.
- Phase 3. Analyse comparative et développement de règles** (jours 8–11). Classification des alertes (vrai positif, faux positif, indéterminé). Comparaison quantitative des deux outils. Développement de règles personnalisées pour combler les lacunes identifiées. Validation des règles sur le corpus.
- Phase 4. Rédaction et restitution** (jours 12–14). Rédaction du rapport. Préparation des éventuels rapports de divulgation. Soutenance.

Livrables attendus

- Rapport comparatif détaillé (25–35 pages) incluant la méthodologie, les résultats quantitatifs et l’analyse qualitative.
- Dépôt GitHub contenant les scripts d’automatisation, les règles personnalisées développées, et les données de résultats.
- Matrice comparative CodeQL vs. Semgrep (couverture CWE, performance, ergonomie).
- Le cas échéant, rapport(s) de divulgation responsable pour les vulnérabilités nouvelles.
- Présentation de soutenance.

Projet 4 – Audit de la chaîne d’approvisionnement logicielle : analyse des dépendances et détection de composants vulnérables

Fiche synthétique

Intitulé : Audit de la chaîne d’approvisionnement logicielle : analyse des dépendances et détection de composants vulnérables

Mots-clés : supply chain security, SBOM, SCA, OSV, Syft, Gype, dépendances, CVE

Nombre d’étudiants : 3 à 5

Durée : 1 mois (2–3 semaines)

Prérequis : Programmation, gestion de paquets (npm, pip, Maven), notions de sécurité applicative

Contexte et motivation

Les attaques ciblant la chaîne d’approvisionnement logicielle (*software supply chain attacks*) sont devenues l’une des menaces les plus critiques en cybersécurité. L’incident SolarWinds (2020), la vulnérabilité Log4Shell dans Apache Log4j (2021), et les attaques par *dependency confusion* ont démontré que la sécurité d’une application ne dépend pas uniquement de son propre code, mais aussi de l’ensemble de ses dépendances directes et transitives. Un projet logiciel moderne peut embarquer des centaines de bibliothèques tierces, chacune pouvant contenir des vulnérabilités connues (CVE).

La notion de *Software Bill of Materials* (SBOM) — un inventaire structuré de tous les composants d’un logiciel — est devenue un standard industriel (formats SPDX, CycloneDX) et une exigence réglementaire dans certains pays (Executive Order 14028 aux États-Unis). Des outils open source comme Syft (génération de SBOM), Gype (scanner de vulnérabilités) et la base de données OSV (*Open Source Vulnerabilities* de Google) permettent d’automatiser cet audit.

Ce projet propose aux étudiants de réaliser un audit complet de la chaîne d’approvisionnement de plusieurs projets logiciels, en générant leurs SBOM, en identifiant les composants vulnérables, et en évaluant la “dette de sécurité” liée aux dépendances.

Objectifs du projet

- O1.** Sélectionner un corpus de 10 à 15 projets open source populaires couvrant plusieurs écosystèmes (npm/JavaScript, pip/Python, Maven/Java, Go modules).
- O2.** Générer le SBOM de chaque projet à l’aide de Syft (formats SPDX et CycloneDX).
- O3.** Scanner chaque SBOM avec Gype pour identifier les dépendances affectées par des CVE connues. Croiser les résultats avec la base OSV de Google.
- O4.** Analyser la structure du graphe de dépendances : profondeur, largeur, dépendances transitives, composants les plus partagés (*most depended-upon*).
- O5.** Évaluer la “dette de sécurité” : nombre de CVE ouvertes, sévérité (CVSS), ancienneté, disponibilité de correctifs.
- O6.** Proposer une méthodologie d’audit reproductible et un tableau de bord de suivi.

Méthodologie

Phase 1. État de l’art et constitution du corpus (jours 1–3). Revue des travaux sur la sécurité de la chaîne d’approvisionnement logicielle. Sélection des projets cibles selon des critères de diversité (langage, taille, domaine applicatif). Installation de Syft, Gype et de l’API OSV.

- Phase 2. Génération des SBOM et scan de vulnérabilités** (jours 4–7). Génération automatisée des SBOM pour chaque projet. Exécution de Grype sur chaque SBOM. Interrogation de l'API OSV pour chaque composant identifié. Collecte des données dans une base structurée.
- Phase 3. Analyse du graphe de dépendances et de la dette de sécurité** (jours 8–11). Construction et visualisation des graphes de dépendances. Calcul des métriques de dette de sécurité. Identification des composants critiques (*single points of failure*). Analyse comparative entre écosystèmes.
- Phase 4. Rapport, tableau de bord et restitution** (jours 12–14). Rédaction du rapport. Développement d'un tableau de bord simple (script Python ou application web légère) pour visualiser les résultats. Soutenance.

Livrables attendus

- Rapport d'audit complet (25–35 pages) couvrant la méthodologie, les résultats par projet et par écosystème, et les recommandations.
- Collection de SBOM générés (formats SPDX et CycloneDX).
- Base de données des vulnérabilités identifiées avec métadonnées (CVE, CVSS, composant affecté, correctif disponible ou non).
- Tableau de bord de visualisation (prototype fonctionnel).
- Pipeline d'audit automatisé (scripts reproductibles).
- Présentation de soutenance.

Compétences développées

Ce projet permet aux étudiants de développer des compétences directement valorisables en entreprise :

- Maîtrise des standards SBOM (SPDX, CycloneDX) de plus en plus exigés par les régulateurs.
- Utilisation d'outils de *Software Composition Analysis* (SCA) déployés dans les pipelines CI/CD industriels.
- Compréhension des mécanismes d'attaque de la chaîne d'approvisionnement.
- Capacité à évaluer et prioriser les risques liés aux dépendances logicielles.

Récapitulatif et modalités pratiques

TABLE 2 – Récapitulatif des quatre projets tutorés.

#	Intitulé	Outils	Axe
1	Cartographie de l'espace IP burkinabè et évaluation de l'exposition des services réseau	ZMap, LZR, ZAnnotate	Réseau
2	Audit de l'écosystème HTTPS/TLS des serveurs web burkinabè	ZMap, ZGrab2, ZLint	Réseau
3	Détection automatisée de vulnérabilités dans des projets open source par analyse statique	CodeQL, Semgrep	Logiciel
4	Audit de la chaîne d'approvisionnement logicielle : dépendances et composants vulnérables	Syft, Grype, OSV	Logiciel

Modalités d'évaluation

Chaque projet sera évalué selon les critères suivants :

Critère	Pondération
Rapport technique (rigueur, complétude, rédaction)	40 %
Qualité des livrables techniques (code, données, outils)	25 %
Soutenance orale	20 %
Organisation du travail en équipe (suivi, répartition)	15 %

Ressources et références

- Suite ZMap : <https://zmap.io/> – Documentation, publications et code source.
- CodeQL : <https://codeql.github.com/> – Documentation officielle et tutoriels.
- Semgrep : <https://semgrep.dev/> – Règles communautaires et documentation.
- Syft / Grype : <https://github.com/anchore/syft> et <https://github.com/anchore/grype>.
- OSV (Open Source Vulnerabilities) : <https://osv.dev/> – Base de données Google.
- AFRINIC WHOIS : <https://www.afrinic.net/> – Données d'allocation IP pour l'Afrique.
- NVD (National Vulnerability Database) : <https://nvd.nist.gov/>.
- CWE (Common Weakness Enumeration) : <https://cwe.mitre.org/>.